

Compiled Component Profiles and Event-Driven Port Quotients

Exact local acceleration toward quasi-polynomial unknot recognition

Research contribution prepared for the ProveIt project
Mathematical development and reference software prepared with ChatGPT

8 October 2026

Repository baseline: 1fab6d945be5

Abstract

The maintained ProveIt unknot-recognition project already contains exact weighted interval-orbit computations, sparse component incidence, and componentwise normal-surface statistics. We develop a different continuation: compile a supplied interval relation once into the multiplicities of its source-atom profiles, then reuse this census for additive observations and monotone full-port identifications. A capacity-aware scalar encoding recovers the complete profile census through one weighted orbit call. A multiplicity-aware incidence graph computes subsequent quotients without expanding components, and its incremental implementation scans each support edge at most once during a monotone phase.

The central structural result is a sharp height bound: on m fixed nonempty source atoms, at most $2m - 1$ full-port coning operations can change the component partition. These operations form a commutative idempotent action. Consequently a straight-line program containing binary repetitions admits compressed final-state and prefix evaluation, a first-threshold search with at most $g + 1$ state trials for g grammar nodes, and enumeration of every count-changing step in $O((h + 1)g)$ trials, where $h \leq 2m - 1$. Large step indices remain binary integers; their bit costs are included.

We also prove an exact observational-equivalence characterization and give a four-point obstruction to extending the cache to arbitrary pointwise attachments. The package includes independent graph and grammar replay, 25 passing test methods, 9,338 exhaustive finite coning cases, randomized checks, and controlled compressed-program measurements. Native weighted-AHT integration is supplied as an adapter and an explicit unrun gate, not claimed as an executed production benchmark. These are local algorithmic results; no general quasi-polynomial unknot-recognition bound is established.

Contents

1	Research question and precise status	3
2	Input model, source atoms and bit costs	4
3	One-time compilation into component profiles	5
4	Exact quotient action on the compiled census	7
5	A sharp height bound for fixed-port phases	10
6	Compressed programs, threshold search and complete event histories	11
7	Certificates, source binding and independent replay	14
8	End-to-end cost of one fixed-source phase	16
9	Executed validation and controlled measurements	18
10	Further research questions and proposed next experiments	20
11	Conclusion	23
A	Artifact map and reproduction	23
B	Minimal usage example	24

1 Research question and precise status

1.1 What is being accelerated

The input to the local problem is a finite equivalence relation encoded by isometries between integer intervals. The integer universe may be enormous, while the number of interval pairings and their endpoint bit lengths are small. A connected normal surface component can be represented by an orbit of such a relation. The immediate computational objective is to avoid repeatedly rediscovering or transporting the same component information when many observations share one fixed source.

An important distinction is between a *new weight query* and a *new relation*. A reusable unweighted reduction trace avoids a fresh orbit search, but a new weight query still performs transport through that trace. On the other hand, adding genuine identifications changes the relation, so blindly reusing the old final histogram is unsound. We identify an intermediate interface: new additive observations may be constant on a fixed source partition, and new relations may collapse whole unions of its atoms. For this interface, a single compiled table is both sufficient and closed under all subsequent operations.

Here a *cone* is an equivalence-relation operation, not a statement that a particular topological cone or manifold attachment is valid. It identifies every point in a specified union. A gluing that identifies one point in an interval with one corresponding point in another interval is a different operation. Keeping these meanings separate is essential to every result below.

1.2 Relationship to the inspected repository

The repository was inspected through its GitHub interface, then pinned at commit `1fab6d945be52857cce48b6459075e6194622001`. The scope is `Topology/UnknotRecognition`; the inspected source paths and blob hashes are recorded in `integration/source_audit.json`. Earlier overview text lagged behind the newer modules. The present article therefore treats the newer native sparse and weighted implementations as prior work, rather than re-presenting their results as new contributions [4, 5, 6].

In particular, the baseline already has a sparse positive component-incidence interface, a generic weighted orbit histogram, independent weighted replay, and normal-surface component queries. It also supports reusing a verified unweighted trace for another weighted query. The contribution here is the compiled profile interface, its exact quotient action, the sharp height and compressed-history bounds, and the corresponding additive reference package. It is not a replacement implementation of the underlying orbit algorithm.

The source was readable, but a runnable native checkout was not available in the execution environment. Thus the adapter’s function signatures were checked against the pinned files, while its native execution gate remains `NOT_RUN`. The reference implementation and its independent finite oracles were executed. No maintained production module is modified by this package, no complete maintained test-suite result is asserted, and no whole-knot speedup is reported.

1.3 Classical input and the global target

Weighted orbit counting is classical. Agol–Hass–Thurston give a polynomial algorithm in the pairing count, weight-run count, weight dimension, and binary numerical sizes; their output uses compressed constant-weight blocks, not a separate expanded record for every represented orbit [1, Theorem 16]. We use this theorem as a compilation primitive. Union by rank with path compression supplies

the standard amortized set-union bound [3]. Neither ingredient is claimed as a new algorithmic theorem here.

Lackenby’s 2026 hierarchy algorithm is relevant to the larger programme. Section 9 separates iteration counting from step costs and does not give a complete runtime estimate. Its Proposition 9.1 bounds iterations using hierarchy length and pattern complexity; the refinement in Section 10 bounds hierarchy length linearly in initial q -complexity [2]. This does not supply the missing end-to-end quasi-polynomial implementation bound.

Our narrow question is therefore concrete: after a relation has been encoded and its future full-port operations described compactly, can the component observer process the entire phase in time polynomial in that description, rather than in its expanded number of operations? The answer is yes. The much stronger question—whether all operations needed by an unknot recognizer can be organized into a controlled number of such phases—is not answered here.

1.4 Main statements at a glance

The central theorem package has four parts. First, a carry-free scalar weight recovers the full profile census with binary width determined by the individual atom capacities. Second, a small graph computes the exact action of a collection of full-port cones, including multiplicity collapse. Third, the algebra of these cones has height at most $2m - 1$, sharply. Fourth, this algebra permits direct navigation of compressed operation programs, including all effective event positions.

The results are presented with proofs, not inferred from experiments. The experiments instead test the implementation, expose interface mistakes, and measure the particular kernel improvements. The article makes no assertion of priority over all literature on finite partition algebras or compressed program evaluation. Its contribution is the derived theorem package and implementation for this precisely specified ProveIt observer interface.

2 Input model, source atoms and bit costs

Let $X = \{0, \dots, N - 1\}$. An interval pairing has inclusive domain $[a, b]$ and range $[c, d]$, equal lengths, and map either

$$x \mapsto x + c - a \quad \text{or} \quad x \mapsto a + d - x.$$

The equivalence relation E is generated by the pairs $(x, f(x))$ for all supplied pairings f . The maps are not expanded point by point. There are k pairings and every endpoint has at most $B = \max(1, \lceil \log_2(N + 1) \rceil)$ bits.

Fix cuts

$$0 = c_0 < c_1 < \dots < c_m = N, \quad J_j = [c_j, c_{j+1}) \cap \mathbb{Z}, \quad \ell_j = c_{j+1} - c_j \quad (0 \leq j < m).$$

These half-open *source atoms* partition the whole universe. Gaps between marked ports must be included as atoms; otherwise unmarked components could have zero profile and be lost. For $N = 0$ there are no atoms, no orbits, and a single cut 0. This case is handled separately by the software.

Every future port is a union of complete source atoms. If the initial interface supplies M half-open intervals, their endpoints and the two universe endpoints yield at most $2M + 1$ nonempty atoms. Endpoints of any additional weight functions must also be inserted before compilation. Nothing in this construction authorizes a later cut inside an existing atom.

For arithmetic accounting, let $M(b)$ bound multiplication of b -bit integers; using schoolbook multiplication gives the safe choice $M(b) = O(b^2)$. Additions and comparisons of b -bit counts cost $O(b)$, not unit time. Set-union operation counts are distinguished from these integer costs. We use H for the largest bit length of an expanded program length or a reported event index. A result independent of the *value* of the expanded length can still depend on H .

The implementation is exact Python. Dictionaries give ordinary expected lookup costs; sorting or balanced dictionaries can replace them with polynomial comparison overhead. Cooperative cancellation is supported at the native adapter boundary. The package is not a hostile-input process-memory sandbox, and none of its count bounds is a timeout guarantee.

3 One-time compilation into component profiles

Definition 3.1 (Profile census). For an E -orbit O , define

$$\mathbf{a}(O) = (|O \cap J_0|, \dots, |O \cap J_{m-1}|).$$

The profile census h_E maps a vector $\mathbf{a}$ to the number of orbits with that vector. Only positive entries are stored. Write $s = |\text{supp } h_E|$ and $C = \sum_{\mathbf{a}} h_E(\mathbf{a})$.

A nonempty orbit has a nonzero nonnegative profile. For each atom,

$$0 \leq a_j \leq \ell_j, \quad \sum_{\mathbf{a}} h_E(\mathbf{a}) a_j = \ell_j. \quad (1)$$

The multiplicity $h_E(\mathbf{a})$ is binary. A row can therefore describe 2^{16000} orbits without listing them. Equal profiles do not mean that the orbits are already identified.

3.1 Capacity-aware scalar weights

Choose integer radices $R_j > \ell_j$, set

$$u_0 = 1, \quad u_j = \prod_{i < j} R_i, \quad P = \prod_{j < m} R_j,$$

and assign scalar weight u_j to every point of J_j .

Theorem 3.2 (Carry-free profile compilation). *The sum of these scalar weights on an orbit is*

$$z(O) = \sum_{j < m} a_j(O) u_j \in [0, P).$$

Successive quotient–remainder division by $R_0, R_1, \dots, R_{m-1}$ recovers $\mathbf{a}(O)$ uniquely. Consequently one exact scalar weighted orbit histogram determines the entire profile census, with the same multiplicities. This holds for every interval relation, including relations with reflections and static points.

Proof. For any profile satisfying (1), its j th coordinate is less than R_j . Reducing z modulo R_0 therefore gives a_0 ; subtracting a_0 and dividing by R_0 produces the same representation starting with a_1 . Induction decodes every coordinate. The usual mixed-radix identity $\sum_j (R_j - 1)u_j = P - 1$ bounds z . Conversely encoding the decoded digits reconstructs z , so two distinct profiles cannot collide. Summing scalar weights over an orbit gives exactly the displayed formula regardless of which pairing descriptions generated that orbit. Grouping equal scalar sums therefore groups exactly the same orbits as grouping equal profiles. $\square$

Two implementations are supplied. The **mixed** mode sets $R_j = \ell_j + 1$. The **binary** mode sets

$$b_j = \lceil \log_2(\ell_j + 1) \rceil, \quad R_j = 2^{b_j}, \quad W = \sum_{j < m} b_j.$$

Binary mode uses fixed-width nonoverlapping fields, with W total bits. A decoded field exceeding ℓ_j is invalid even if it fits its field. Both modes reject negative scalar sums and nonzero leftover high digits.

Proposition 3.3 (Packing width and its limited optimality). *Mixed-radix packing uses the integer range $[0, \prod_j(\ell_j+1))$. Any injective code for the entire box $\prod_j\{0, \dots, \ell_j\}$ needs at least $\lceil \log_2 \prod_j(\ell_j+1) \rceil$ bits in the worst case. Binary mode uses less than m extra bits beyond the unrounded logarithmic bound, and $W \leq mB$.*

Proof. The box has exactly the stated product cardinality. An injective map to b -bit strings needs 2^b at least that cardinality. Mixed-radix encoding is a bijection between the box and its integer range. Rounding each of its m logarithmic contributions upward adds less than m in total. Finally $\ell_j \leq N$ implies $b_j \leq B$. $\square$

This is not an information lower bound for a particular source census, whose support can be far smaller than the full box. Nor is it a claim that packing beats vector arithmetic on every machine. A scalar of width W can make multiplication and division more expensive than a short sparse vector operation. The engineering hypothesis is reduced tuple and dispatch overhead in some weighted-replay workloads; native measurements are still required before selecting a default mode.

Example 3.4 (Nonuniform capacities). Take lengths $(2, 5, 1)$. Mixed mode has radices $(3, 6, 2)$ and units $(1, 3, 18)$. The profile $(2, 4, 1)$ has code 32. Decoding gives remainders 2, 4, 1. The full capacity box has 36 elements and needs six bits. Binary mode has widths $(2, 3, 1)$, also six bits. Using the common universe bound $N = 8$ for every field would instead allocate twelve bits. The example illustrates capacity savings, not a measured native timing claim.

3.2 Why the compiled table has polynomial encoded size

Weighted AHT applied to the m basis-vector weight runs gives a compressed profile output in polynomial time in k, m, B . Applying it instead to the scalar weights above has one coordinate and weight bit sizes $O(W + \log m)$, again polynomial in k, m, B [1]. Decoding and coalescing its polynomial number of output runs yields h_E . Thus the fact that s is polynomially bounded for interval-encoded input follows from classical weighted AHT; it is not a new sparsity assumption introduced to make the theorem work.

The article deliberately leaves the underlying polynomial as $P(k, m, B)$ rather than guessing a small exponent for the maintained code. In particular, a polynomial theorem does not mean that repeated compiled queries are costless or that one scalar call has constant-size arithmetic. The software records native statistics without interpreting an AHT cycle allowance as a bound on all bit operations.

3.3 Reusing the table for observations

Let $w_j \in \mathbb{Z}^d$ be a fixed weight vector on atom J_j . Define

$$A_w(\mathbf{a}) = \sum_{j < m} a_j w_j.$$

Then the component-weight histogram is

$$H_w(v) = \sum_{\mathbf{a}: A_w(\mathbf{a})=v} h_E(\mathbf{a}). \quad (2)$$

This formula permits signed weights, including Euler-characteristic charges when their geometric ownership was previously validated. It costs $O(smd)$ integer multiplications and additions plus grouping, with no orbit search and no trace transport. Packing is used only for nonnegative source counts; signed weights are applied after decoding.

More generally, any explicitly computable predicate F on a profile gives $\sum_{\mathbf{a}: F(\mathbf{a})} h_E(\mathbf{a})$. Port incidence is recovered by checking whether $\sum_{j \in A} a_j$ is positive. Several observations may be performed together as a vector map, preserving their joint distribution. Computing separate marginal histograms and later combining them generally loses correlations; the joint profile table avoids that loss.

Example 3.5 (Why marginal caches do not suffice). A table with profiles $(1, 1)$ and $(2, 2)$, each of multiplicity one, has the same two coordinate marginals as the table with profiles $(1, 2)$ and $(2, 1)$. Both have atom lengths $(3, 3)$. The observation $a_0 - a_1$ distinguishes them: the first gives two zeroes and the second gives -1 and 1 . Full joint profiles, rather than independent atom marginals, are needed for universal observation reuse.

4 Exact quotient action on the compiled census

4.1 Full-port coning and its incidence graph

For $A \subseteq \{0, \dots, m-1\}$, write $J_A = \bigcup_{j \in A} J_j$. The operation $\text{Cone}(A)$ replaces E by the equivalence closure of $E \cup (J_A \times J_A)$. Empty A has no effect. A singleton atom set A need not be a no-op: it identifies all points within that one atom.

Suppose a collection $\mathcal{A}$ of cone groups is applied. Construct a graph with one vertex for each profile type and one vertex for each active atom $j \in \bigcup \mathcal{A}$. Join a type $\mathbf{a}$ to an active atom j when $a_j > 0$. Join the atom vertices within each $A \in \mathcal{A}$, using a star or path. Let $\mathcal{K}$ denote the connected components of this graph that contain a type vertex, and let T be the set of its type vertices.

Theorem 4.1 (Multiplicity-aware quotient formula). *The profile census after applying $\mathcal{A}$ is obtained as follows:*

- (i) *Every untouched type $\mathbf{a} \notin T$ retains its original multiplicity $h_E(\mathbf{a})$.*
- (ii) *Each graph component $K \in \mathcal{K}$ contributes exactly one orbit, with profile*

$$b(K) = \sum_{\mathbf{a} \in K} h_E(\mathbf{a}) \mathbf{a}.$$

- (iii) *Equal resulting profiles are coalesced by adding multiplicities.*

In particular the orbit count is

$$C_{\mathcal{A}} = \sum_{\mathbf{a} \notin T} h_E(\mathbf{a}) + |\mathcal{K}|. \quad (3)$$

The resulting census has at most s distinct rows.

Proof. First consider a touched type $\mathbf{a}$ and an incident active atom j . Each of its $h_E(\mathbf{a})$ original orbits contains a point in J_j . Some applied cone contains the entire atom, so all those points, and therefore all copies of that type, become equivalent. This justifies using one type vertex instead of $h_E(\mathbf{a})$ vertices. It also explains why the factor $h_E(\mathbf{a})$ must occur in $b(K)$.

An edge from a type to an atom means that the corresponding orbit meets the coned set. Edges within a cone group express the same full identification. Thus any two type vertices in one graph component become a single orbit. Conversely, an identification path between original orbits alternates old orbit connections and cone connections. Replacing its old orbit occurrences by their types and its cone occurrences by their atom vertices gives a path in this graph. No two distinct graph components can therefore merge. An untouched original orbit meets no coned set and retains its identity.

All original points in the copies of a touched type enter the same new orbit. Their atom counts add, giving $b(K)$. This proves the census formula and (3). If t types are touched, at most t graph components contain types, while at most $s - t$ untouched rows remain. Merging equal outputs can only reduce this number, proving the row bound. $\square$

No extra orbit is assigned to a purely virtual atom component. Because every atom is nonempty and the profile table covers its mass, every active atom in a valid complete table actually meets a type. The explicit condition in the theorem remains useful in the implementation and its checker.

Example 4.2 (A binary population collapses once). Let every atom have length q , and suppose the source consists of q parallel components, one point in each atom. Then $h_E((1, \dots, 1)) = q$. Coning any single atom produces one component with profile $(q, \dots, q)$, not q components and not one component of profile $(1, \dots, 1)$. The transformation uses one multiplication per coordinate; there is no loop of length q .

4.2 The exact information retained by the table

Theorem 4.3 (Relative completeness and minimality). *For two equivalence relations E, F on the same partitioned set X , the following conditions are equivalent:*

- (i) $h_E = h_F$.
- (ii) *There exists a bijection of X preserving each atom setwise and sending the E -orbits bijectively to the F -orbits.*
- (iii) *Every finite sequence of full-port cones, followed by any atom-constant additive observation, has the same component-weight histogram for the two sources.*

Thus the census is a complete invariant for this observer interface, and any exact summary supporting every such observation must distinguish unequal censuses.

Proof. If the censuses agree, pair the orbits of each profile type. For each paired orbit and each atom, choose a bijection between their equally sized intersections. Their union is a bijection of X with the required properties. Conversely such a bijection preserves every orbit profile, proving (i) $\Leftrightarrow$ (ii).

Every full-port cone is unchanged by an atom-preserving permutation, and an atom-constant weight is unchanged as well. Therefore the bijection remains an isomorphism after any cone sequence and preserves its weight histogram. This proves (ii) $\Rightarrow$ (iii). Finally take no cones and use the scalar

weight of theorem 3.2. Its histogram encodes the entire census, so (iii) implies (i). The last assertion follows by using this same observation to distinguish any two unequal tables. $\square$

The bijections in this proof may be arbitrary permutations inside atoms; they are not asserted to be interval isometries or geometric homeomorphisms. The theorem identifies exactly the information forgotten by the observer. It does not imply that the full three-dimensional objects represented by E and F are equivalent.

4.3 A small obstruction to more general attachments

Proposition 4.4 (Profiles do not determine partial attachment outcomes). *A complete source-profile census is insufficient to compute the effect of an arbitrary additional interval pairing, even when that pairing has singleton domain and range.*

Proof. Let $X = \{0, 1, 2, 3\}$, with atoms $\{0, 1\}$ and $\{2, 3\}$. Set

$$E = \{\{0, 2\}, \{1, 3\}\}, \quad F = \{\{0, 3\}, \{1, 2\}\}.$$

Both have $h((1, 1)) = 2$. Add the singleton pairing $0 \sim 2$. It is already present in E , leaving two orbits. In F it connects the two original orbits, leaving one. The input summaries are identical and the requested additional pairing is identical, but the answers differ. $\square$

A related error would be to refine an atom after compilation and infer its subatom profiles from the old table. Splitting the first atom in the example into $\{0\}$ and $\{1\}$ immediately requires information that the old table forgot. Refining every point would avoid this particular loss, but can require N atoms, exponential in B . This is a genuine representation barrier, not a reason to abandon richer compressed attachment descriptions.

4.4 Incremental evaluation without repeated graph construction

The implementation maintains disjoint-set nodes for the s original types and the m atom vertices. Atom nodes are initially inactive and virtual. A type not yet touched represents $h_E(\mathbf{a})$ separate real components, although only one dormant node is stored for it.

When atom j is activated for the first time, scan its adjacent types. If a type is still inactive, replace its population by one active node and decrease the count by $h_E(\mathbf{a}) - 1$. Join each type to the atom node. Joining two roots that both contain real components decreases the count by one; joining a virtual root to a real root does not. Once all atoms in the cone group are activated, join their nodes together. Duplicate activations never scan adjacency again.

Theorem 4.5 (Online amortization). *Let*

$$e = \sum_{\mathbf{a} \in \text{supp } h_E} |\{j : a_j > 0\}|.$$

After an $O(sm)$ scan of the dense profile coordinates, a monotone sequence of cone calls with u total supplied atom occurrences uses

$$O((s + m + e + u)\alpha(s + m) + u \log(m + 1))$$

index-level operations, including canonicalization of input groups. Each type population is collapsed at most once and each incidence edge is scanned at most once. Count arithmetic uses B -bit integers. Materializing a complete quotient census is a separate $O(sm)$ arithmetic operation and is not hidden inside the set-union bound.

Proof. The activation rule implements exactly the two stages of theorem 4.1: collapse all copies of a touched type, then merge connected touched types. A type cannot already belong to a disjoint-set component when its population is first activated, because edges involving that type are inserted only by activation. The “real root” indicator therefore accounts for every decrease exactly once.

Each atom is activated once, so each incident edge is visited once. There are at most s first type activations. Group processing adds at most u atom-node union attempts. Union by rank and path compression give the stated amortized bound, with the sorting term paying for canonical atom sets. The count stays between zero and the initial $C \leq N$, so it uses at most B bits. Census materialization visits each original type and adds $h_E(\mathbf{a})\mathbf{a}$ to its current root, then coalesces equal rows. This last step uses vector arithmetic and sorting rather than only index operations. $\square$

The prototype stores the original profile table immutably. Branching a threshold search clones the small mutable set-union state but shares that table and its adjacency lists. It never substitutes a modified census for the source certificate. This separation both prevents rebinding mistakes and makes the exact cost of repeated state trials visible.

5 A sharp height bound for fixed-port phases

5.1 The algebra of active subpartitions

The effect of a cone sequence on the source atoms has a compact canonical description. Record the active atom set A and partition it into blocks, where two active atoms lie in the same block when the applied cone groups connect them. Each block means that all points in the union of its atoms are identified. Atoms outside A impose no new relation. This is an *active subpartition*, not just an ordinary partition of all m atoms. In particular, an active singleton block must remain visible.

The join $S \vee T$ of two active subpartitions has active set the union of their active sets and equivalence blocks generated by both collections of blocks. It is computed by ordinary finite set union. The empty collection is the identity.

Lemma 5.1 (Commutative idempotent action). *Joining summaries agrees with composing their cone actions. Thus*

$$(S \vee T) \vee U = S \vee (T \vee U), \quad S \vee T = T \vee S, \quad S \vee S = S.$$

These identities hold for every initial relation E , not only for the initial discrete relation.

Proof. Let R_S be the union of the full relations on the point sets represented by the blocks of S . Applying S and then T gives the equivalence closure of $E \cup R_S \cup R_T$. Its closure depends neither on the order nor on repeated copies of these generating relations. Two atom groups intersect precisely when their point unions share an atom, so the connected blocks of the join generate the same full relations. Associativity, commutativity and idempotence follow. $\square$

For example, coning atom 0, then atom 1, does *not* mean that atoms 0 and 1 are connected: the summary has two singleton blocks. Coning $\{0, 1\}$ instead has one two-element block. Dropping singleton blocks would incorrectly erase the first two operations.

Theorem 5.2 (Sharp fixed-atom height). *For $m \geq 1$, a sequence of full-port cones on m fixed nonempty atoms contains at most $2m-1$ steps at which the component partition changes. Equivalently, its component count has at most $2m-1$ strict drops. The bound is attained for every m . For $m = 0$ there are no changes.*

Proof. For an active subpartition with a active atoms and b blocks, set

$$\Phi = 2a - b.$$

The empty summary has $\Phi = 0$. A nonempty summary has $0 \leq \Phi \leq 2m-1$, since $a \leq m$ and $b \geq 1$.

Suppose a new nonempty cone group contains k previously inactive atoms and meets q old active blocks. Its update replaces those q blocks by one block also containing the k new atoms. Consequently

$$\Delta\Phi = 2k + q - 1.$$

When the summary really changes, either $k \geq 1$, or $k = 0$ and $q \geq 2$; in both cases $\Delta\Phi \geq 1$. If $k = 0$ and $q = 1$, the summary is unchanged. An empty cone is also unchanged.

By lemma 5.1, an unchanged summary cannot change the relation on points, whatever the initial E . Every actual partition change therefore uses at least one unit of the bounded potential, giving the result. A strict coarsening of a finite partition decreases its number of blocks, so partition changes and strict count drops are equivalent here.

For sharpness, take $2m$ initially isolated points and m atoms of size two. Cone each singleton atom in turn, producing m strict drops. Then cone $\{0, j\}$ for $j = 1, \dots, m-1$, producing another $m-1$ strict drops. There are exactly $2m-1$ changes. $\square$

This potential is qualitatively different from a large decreasing binary counter. It has height linear in the number of fixed atoms, independent of their capacities, component multiplicities, and the number of redundant instructions. The hypothesis “fixed” cannot be omitted. An operation that introduces a new atom or removes an old identification changes the state space in which the potential is measured.

6 Compressed programs, threshold search and complete event histories

6.1 Straight-line program contract

A program is an acyclic list of g nodes. A node is one of

$$\text{Cone}(A), \quad \text{concat}(P, Q), \quad \text{power}(P, p), \quad p \in \mathbb{N}.$$

References point strictly to earlier nodes. A root selects the requested program. A cone is one instruction, including an empty cone. Concatenation adds lengths and power multiplies a length by p . A zero power is an empty program. Exponents and lengths are exact binary integers.

This language contains fixed atom names in its leaves. It does not contain an instruction of the form “shift this port by the current iteration number”, an unknown attachment selected adaptively

from new geometry, or a relabeling hidden inside a power. Such languages require additional semantics and are outside the following proofs.

Let $L(P)$ be expanded length and $S(P)$ the active-subpartition summary. Bottom-up compilation uses

$$\begin{aligned} L(\text{Cone}(A)) &= 1, & S(\text{Cone}(A)) &= \{A\} \quad (A \neq \emptyset), \\ L(PQ) &= L(P) + L(Q), & S(PQ) &= S(P) \vee S(Q), \\ L(P^p) &= pL(P), & S(P^p) &= \begin{cases} \emptyset, & p = 0, \\ S(P), & p > 0. \end{cases} \end{aligned}$$

The summary of an empty cone is empty, not an empty block.

Proposition 6.1 (Final-state and prefix compilation). *Every summary has at most m atom entries. All node summaries and expanded lengths can be computed in polynomial time in the explicit grammar, m , and the exponent bit lengths. For a supplied $0 \leq t \leq L(P)$, the summary of the first t expanded instructions can be computed by visiting at most one descending grammar path and joining at most g stored summaries.*

Proof. Each active subpartition consists of disjoint blocks, so its total atom entries are at most m . The recursive summary rules are correct by lemma 5.1; backward references make their bottom-up evaluation well-defined.

For a prefix of PQ , descend into P if $t \leq L(P)$. Otherwise include $S(P)$ and descend into the first $t - L(P)$ instructions of Q . For a prefix of P^p , descend into P if $t < L(P)$. If $t \geq L(P) > 0$, the prefix already contains one full copy of P . Its remaining partial or complete copies add only relations already present in $S(P)$, so its entire summary is $S(P)$. A full-node prefix returns its stored summary immediately. Every descent follows a strict backward reference and therefore has length at most g . Zero-length nodes are handled by their empty summary and never cause division by zero. $\square$

With union by rank and path compression, normalizing and joining explicit blocks costs $O(m \log(m + 1))$ index operations using the supplied canonical sorting conventions. If a is the number of atom occurrences in the explicit leaves, an adequate compilation bound is

$$O((a + gm) \log(m + 1)) \text{ index operations, plus } O(gM(H)) \text{ bit operations.}$$

Indeed $H \leq g + \sum \text{bits}(p)$ over the power nodes is a safe upper bound: a concatenation adds at most one bit on a dependency path and a power adds at most the bit length of its exponent. The input itself and the output positions therefore account for all large integers.

6.2 Direct first-threshold search

Write $C(t)$ for the component count after the first t instructions. Since cones only coarsen partitions, $C(t)$ is nonincreasing. Prefix bisection would locate the first t with $C(t) \leq K$ in $O(\log(L(P) + 1))$ exact prefix evaluations. This is already polynomial in H , but its evaluation count can be much larger than the grammar size.

Theorem 6.2 (Grammar-directed first threshold). *Given a compiled profile table and a g -node fixed-port program, the first prefix whose component count is at most an integer K can be found*

with at most $g + 1$ quotient-state trials. The answer is 0 if the initial count qualifies and is *None* if the final count does not qualify. The trial count is independent of the numerical expanded length; arithmetic on its H -bit indices is still charged.

Proof. First test the initial count and, if necessary, the root's complete summary. If neither provides a terminal answer, maintain a current quotient state, a node, and an expanded offset such that the target is not yet reached but will be reached somewhere inside that node.

At a concatenation PQ , apply $S(P)$ to a clone of the current state. If its count qualifies, descend into P without changing the original state. Otherwise adopt the trial state, add $L(P)$ to the offset, and descend into Q . At a positive power P^p , descend into its first copy of P : after that copy the state already has the full effect of all p copies by idempotence. Thus a target attained by the power is attained in its first copy. A zero power cannot satisfy the maintained invariant. At a cone leaf, apply the cone and return the offset plus one.

The invariant is preserved in every case. Each step follows a strictly backward grammar reference, so there are at most g visited nodes. Only concatenations need a new trial, besides the initial full-root test. This gives the stated bound. Monotonicity and the left-first descent prove that the returned prefix is the first one, not merely one qualifying prefix. $\square$

Example 6.3 (A huge delayed threshold). Let q parallel components meet each of eight atoms once. Let I be the empty cone and $A = \text{Cone}(\{3\})$. The five-node program

$$P = (I^{2^b} A)^{2^b}$$

has expanded length $2^{2b} + 2^b$. Its first count at most one occurs at $t = 2^b + 1$. The direct algorithm needs two state trials for every b . The included prefix-bisection baseline uses $2b + 2$ evaluations on this family. Reading $b + 1$ exponent bits and writing the event position remain necessary; the theorem eliminates repeated state evaluation, not bit complexity itself.

6.3 Enumerating every effective event

Theorem 6.4 (Complete event-driven history). *For a g -node fixed-port program, all expanded indices at which its component partition changes can be enumerated without expanding repetitions. If there are h such events, the algorithm uses $O((h + 1)g)$ quotient-state trials and returns h records of the form $(t, C(t - 1), C(t))$. In particular $h \leq \max(0, 2m - 1)$ and the entire history is polynomial in the encoded input parameters.*

Proof. Traverse the grammar from the root with a current quotient state and an expanded offset. At each encountered node, tentatively apply its complete summary. If the count is unchanged, skip the entire node. This is sound not just for counts: a coarsening of a finite partition with the same number of blocks is the identical partition. Hence all its intermediate states are also unchanged.

If the count decreases, a cone leaf produces an event and its trial state is adopted. A concatenation is explored left to right, with the right offset increased by the full expanded left length. A positive power is explored only in its first child copy. After that copy no later copy can change the relation, by idempotence.

Every successful nonleaf trial lies on a descending path to at least one reported event. Charge it to the first such event in its explored subtree. Each path has at most g nodes. Each successful concatenation generates at most two child trials, so the skipped siblings add at most a constant

factor. If there are no events, one root trial suffices. This proves $O((h+1)g)$. Correct offsets locate events in the original expanded program, including all skipped copies. Finally apply theorem 5.2. $\square$

The implementation uses an explicit stack, so long grammar dependency chains do not consume Python recursion depth. The tests include 2,500 nodes and event indices beyond ordinary decimal-serialization limits. Hexadecimal transport keeps those limits unchanged rather than modifying interpreter-global settings.

The history theorem is stronger than a final-state shortcut. It recovers all genuine component changes and their exact positions in a potentially enormous schedule. It does not recover an expanded list of redundant operations, whose output size would itself forbid a polynomial bound.

7 Certificates, source binding and independent replay

7.1 Three distinct trust layers

An algebraically well-formed table is not automatically the census of a particular interval relation. The implementation separates three assertions.

Table validity. `Profiles` checks nonempty positive-length atoms, nonzero nonnegative profile vectors, positive integer multiplicities, sorted unique rows, and all coordinate mass equations (1). These checks establish a valid abstract finite census. They do not authenticate the source pairings. Boolean values are rejected where exact integers are required.

Source compilation. The native adapter invokes the maintained scalar weighted producer and wraps its certificate with the original universe, exact atom cuts, and packing mode. Independent verification reconstructs the radix weights, checks the native weighted certificate against the original pairing list, and decodes its authenticated scalar histogram using separate arithmetic code. It returns a newly constructed trusted table. It does not return a Boolean that a caller might mistakenly apply to an unrelated mutable table.

Quotient or program claim. Given an authenticated table, independent graph replay constructs the quotient without calling the incremental engine. Independent grammar replay validates backward references and computes summaries through a separate graph-based join rather than the producer’s set-union code. These checkers establish the subsequent claim relative to that source table. They do not turn an unauthenticated table into a knot certificate.

7.2 A quotient certificate need not reproduce union-find decisions

The graph in theorem 4.1 depends only on the original census and cone groups. A verifier can reconstruct it from scratch and compare the complete claimed output census. There is no need to trust the producer’s choice of roots, compression history, activation order, or internal count updates. Equal count totals alone are insufficient: different profiles can have the same number of components. The checker therefore compares the entire canonical table when a census is claimed.

A bounded literal oracle supplies a further level of independence in the finite tests. It explicitly builds the relation on points, adds actual full-port equivalences, and counts each component’s atom

intersections. It neither uses profile-type collapse nor relies on the graph quotient formula. The literal oracle has a point cap and is not used on huge multiplicities.

7.3 First-threshold certificates

A compact claim consists of the source table, validated grammar, target K , and claimed position t . A trusted source table may be referenced by an already checked compilation certificate rather than recopied.

Proposition 7.1 (Two adjacent prefixes certify the first threshold). *For $1 \leq t \leq L(P)$, the equations*

$$C(t-1) > K \geq C(t)$$

certify that t is the first qualifying prefix. A claim $t = 0$ is checked against the initial count. A claim that the threshold is never attained is checked against the final count.

Proof. Every earlier prefix has count at least $C(t-1)$ by monotonicity, so none qualifies. The second inequality shows that t does. The endpoint cases are the same monotonicity argument. $\square$

The verifier reconstructs both prefixes independently using proposition 6.1, then performs graph quotients. No list of the first t instructions is required. The bit length of t is part of the certificate size. An out-of-range prefix, forged source cut, negative exponent, forward grammar reference, or changed cone group is rejected.

7.4 Certifying the complete event history

Suppose events are claimed at $0 < t_1 < \dots < t_h \leq L(P)$ with before/after counts b_i, a_i . Check

$$C(t_i-1) = b_i > a_i = C(t_i)$$

for every event. Also check that $b_1 = C(0)$, that $b_{i+1} = a_i$, and that $C(L(P)) = a_h$. For an empty list, require $C(L(P)) = C(0)$.

Proposition 7.2 (Plateau completeness). *The preceding checks prove both correctness of every listed event and absence of every omitted count-changing event.*

Proof. The adjacent-prefix comparisons validate the listed drops. Between two listed drops, the checked endpoints have equal counts. A nonincreasing integer-valued sequence with equal endpoint values is constant throughout that interval. The same argument handles the initial and final gaps. $\square$

This is stronger than checking only that the final count is correct. A producer that silently omits an interior event will violate one of the plateau equalities. The tests delete the first and last events from sharp examples, mutate event indices and counts, and disable producer functions while executing the independent checker.

7.5 What is not certified

The full source binding reaches an interval relation, not automatically a manifold homeomorphism type or knot diagram. Likewise, summing an old Euler charge over a merged equivalence class gives a sum of old charges. It does not automatically give the Euler characteristic of a topological object formed by attaching new cells. Such cells need correctly signed ownership charges and a separate geometric proof. Orientation, cyclic order, slope, and pointwise attachment maps are not supplied by the profile table.

The optional native adapter also does not claim an independent proof of the classical AHT theorem. It relies on the maintained local trace verifier for that layer. The added checker is independent of the new compilation and quotient producers at the layers it actually verifies. Independence is thus an explicit dependency boundary, not the claim that no code or mathematical lemma is shared anywhere.

8 End-to-end cost of one fixed-source phase

8.1 Compilation, observations and monotone updates

Write $P(k, m, B)$ for a valid polynomial bound on source-profile compilation through weighted AHT, including decoding and exact source validation. Let s be the number of profile rows, e its support incidence count, and let Q later observations each have dimension at most d and weight entries of at most D bits. A conservative batch bound is

$$P(k, m, B) + O(QsmdM(B + D + \log(m + 1))) \quad (4)$$

plus canonical histogram grouping. This replaces Q separate weighted trace transports by one compilation and Q explicit linear maps on the table. It can lose for a small number of low-dimensional queries, because compiling all source coordinates may cost more than answering them directly. No unconditional speed ordering between these strategies is asserted.

For an explicit monotone cone sequence, define the index work

$$U(s, m, e, u) = (s + m + e + u)\alpha(s + m) + u \log(m + 1).$$

The online count bound is $O(smB)$ for reading the already validated dense coordinates, followed by $O(U)$ index operations and at most $O(s + m)$ effective count changes with B -bit arithmetic. A simple all-bit upper bound charges $O(\log(s + m + 1) + B)$ to every index-level operation; it is intentionally conservative. Census output, when requested, adds $O(smM(B))$ arithmetic and sorting. Repeatedly requesting a full dense census is output work, not a violation of the online count bound.

The row bound in theorem 4.1 is useful here. Within a fixed-source phase, quotient profiles never require more than the original s rows, and every coordinate remains at most its original atom capacity. Thus neither histogram length nor coordinate bit size grows through these quotients. This statement concerns the chosen observer, not the size of future geometric descriptions.

8.2 Compressed schedule accounting

Let T_{trial} bound cloning a quotient state and applying one stored summary. A safe choice in index operations is

$$T_{\text{trial}} = O((s + m + e)\alpha(s + m) + m \log(m + 1)),$$

with the preceding count-bit charges. A summary has at most m atom entries, so it never contains an expanded program. The immutable adjacency lists are shared across clones, and their initial $O(sm)$ construction is not repeated by a clone.

After source compilation and grammar preparation, the first-threshold algorithm costs $O(gT_{\text{trial}} + gH)$ in the separated model, while all history events cost

$$O((h+1)g(T_{\text{trial}} + H)), \quad h \leq 2m - 1.$$

Full grammar preparation additionally pays for its explicit leaf occurrences, canonical summary joins, and g binary length computations. There is no factor equal to the numerical expanded length $L(P)$.

The independent checker has its own polynomial cost. Its graph-based summary join can use quadratic atom work, and it may rebuild the grammar data for several prefix queries. These are deliberate simplicity choices, not costs silently charged to the producer. A host measuring a certified operation must include both generation and replay. The timing tables below measure production search, with answer checks outside the timed region, and do not label those times as certified end-to-end native performance.

Corollary 8.1 (Polynomial fixed-source observer). *A binary interval-pairing source, a fixed atom partition, and a finite straight-line program of full-port cones admit exact final census, first-count-threshold and complete count-change history computations in time polynomial in their encoded input sizes. Corresponding algebra-level claims admit polynomial independent replay. A source-bound replay additionally requires a verified weighted orbit certificate.*

Proof. Compilation is polynomial by weighted AHT and theorem 3.2. Its compressed output has polynomial s and hence polynomial $e \leq sm$. All arithmetic uses polynomially many bits, and the grammar's length bits are bounded by its encoded exponents and node count. Apply the preceding bounds and theorem 6.4 and proposition 7.2. $\square$

8.3 How this enters a quasi-polynomial programme

Consider a prospective recognizer divided into R fixed-source phases. For phase i , record its source parameters, actual compilation cost bound, number of observation coordinates, grammar size, and event count. Let $\mathcal{G}$ denote all work outside this observer: source generation, normal-vector discovery, actual topological cuts, verification of attachment semantics, and transfers between phases. The honest total ledger is

$$\begin{aligned} T_{\text{total}} \leq \mathcal{G} + \sum_{i=1}^R [& P(k_i, m_i, B_i) + \text{Cost}(\text{grammar}_i) + \text{Cost}(\text{observations}_i) \\ & + O((h_i + 1)g_i(T_{\text{trial},i} + H_i)) + \text{Cost}(\text{replay}_i)]. \end{aligned} \quad (5)$$

If R and all encoded phase descriptions are at most $2^{O((\log n)^c)}$, for fixed c , then the displayed observer sum is quasi-polynomial in the crossing input size n . This is a consequence of polynomial closure of that size bound, not a proof that the required phases exist. In particular, (5) says nothing favorable about $\mathcal{G}$ without additional theorems.

There is nevertheless a concrete strengthening over per-instruction accounting. A phase with exponentially many redundant fixed-port operations can be processed through its grammar and at most $2m - 1$ effective changes. This removes its expanded instruction count from the ledger. A mere

termination proof based on a binary counter could not make that replacement. The achievement is local and structural: a small-height quotient algebra has replaced a potentially huge operation sequence.

8.4 Why no global recognition theorem follows

Several missing links are mathematically separate. A supplied normal surface can be analyzed without finding a useful one. A valid triangulation can be analyzed without proving that it is the exterior of the input knot. A full-port coning operation can be evaluated without proving that it models a particular hierarchy move. Finally, a bounded-height phase can terminate quickly while the number of phases or the required source refinements is large.

The four-point example proves that one cannot simply place all future attachment maps behind this interface. The fixed-atom potential also resets when fresh atoms or changed source relations are introduced. A useful continuation should therefore target a specific geometric phase in which these semantics can actually be proved, and then measure how often that phase is rebuilt. Treating a local polynomial observer as a complete quasi-polynomial recognizer would skip precisely the hard accounting that (5) makes explicit.

9 Executed validation and controlled measurements

9.1 Evidence scope

All experiments in this section were executed on CPython 3.13.5 on Linux. The scripts, seeds, source SHA-256 hashes, raw samples, and console logs are included. The final deterministic audit passes 25 unittest methods with zero failures and zero errors. A test method may contain thousands of individual checks; the method count must not be confused with the number of fixtures.

The core evidence uses three computational routes: the incremental multiplicity-aware implementation; an independent graph implementation over types and atoms; and, on bounded inputs, a literal pointwise equivalence oracle. Packing tests use a separately named literal weighted backend with the same API shape as the native producer. This tests the adapter’s scalar compilation mathematics, but is not execution of weighted AHT.

Family	Cases	Comparison or property checked
Finite one-cone exhaustive audit	9,338	Every set partition on 1–5 points, every consecutive atom partition, and every atom subset; literal, incremental and graph answers agree.
Random packing sources	500	Both scalar modes agree with literal joint profiles.
Random sequential sources	700	Ten successive cones each: 7,000 complete census comparisons, plus support-edge and activation bounds.
Random grammar sources	400	Every expanded prefix and every count threshold in each bounded fixture; independent event-history replay.
Sharp-height families	16	$m = 1, \dots, 16$ attains exactly $2m - 1$ events, including huge intervening plateaux.

Table 1: Core validation inventory. Counts describe actual executed finite loops or explicit families, not estimates of unrun native coverage.

Additional tests cover a 2^{16000} population on 64 atoms, packing fields with 16,000-bit capac-

ities, a delayed threshold at $2^{16000} + 1$, a 2,500-node dependency chain, empty universes, zero powers, duplicate atom names, malformed grammar references, invalid Boolean integers, exact hexadecimal transport, clone isolation, and transactional input validation. They also exercise the four-point partial-attachment obstruction, signed observations, modified certificate claims, and checker execution with the producer functions disabled.

The exhaustive count can be independently checked combinatorially. For n points there are $\text{Bell}(n)$ set partitions and $\binom{n-1}{m-1}$ consecutive partitions into m atoms, each with 2^m cone subsets. Thus the number of cases is

$$\sum_{n=1}^5 \text{Bell}(n) \sum_{m=1}^n \binom{n-1}{m-1} 2^m = \sum_{n=1}^5 2 \text{Bell}(n) 3^{n-1} = 9338.$$

The audit script also obtains this counter directly by executing the test.

Finite tests cannot prove the asymptotic claims. Their role is to check that the implementation matches the independently proved formulas, particularly where inactive multiplicities or virtual roots make an apparently routine union-find implementation easy to get wrong.

9.2 First-threshold experiment

The fixture has eight atoms and one profile type $(1, \dots, 1)$ of multiplicity 2^{256} . The five-node grammar is the delayed-threshold family from section 6. We compare direct descent with an exact prefix-bisection implementation using the same compiled table and producer prefix evaluator. Each query creates a fresh quotient state. Grammar construction is outside the timed region for both arms.

b	Length bits	Bisection trials	Descent trials	Bisection (ms)	Descent (ms)
64	129	130	2	0.692	0.01018
256	513	514	2	2.794	0.01009
1024	2049	2050	2	11.201	0.00991
4096	8193	8194	2	49.977	0.01053

Table 2: First threshold in $(I^{2^b}A)^{2^b}$. Seven shuffled batched rounds; times are median per-query costs. These are algebra-kernel measurements, not knot timings.

The benchmark uses one warmup per arm and seven measured rounds in a seeded shuffled order. There is a third, identical descent-control arm. For the short descent queries, each timed batch contains 1,000 fresh calls. Bisection uses $\max(1, 1024/b)$ calls per batch. Reported sample times are total batch times divided by the exact call count. Answer assertions and independent source-relative certificate checks are outside these timings.

At $b = 4096$, direct descent uses two state trials and about 0.0105 ms per query; prefix bisection uses 8,194 evaluations and about 49.98 ms. The large timing contrast is expected from this intentionally repetitive algebraic family. It is not a measured speedup over the maintained unknot recognizer, and it does not measure initial AHT compilation. The more portable finding is the exact evaluation count: the direct method depends on the grammar rather than the bit length of its expanded endpoint search range.

An initial unbatched run showed noticeable variability between identical microsecond-scale descent arms. Its raw data are retained as `results/benchmarks_initial_unbatched.json`; they were not silently replaced. The displayed table uses the batched follow-up in `results/benchmarks.json`.

Batched identical-control medians are close, though residual measurement noise remains. No cross-machine significance or confidence interval is inferred from these seven rounds.

9.3 Complete history experiment

For the sharp family, start with m atoms containing two isolated points each. Apply all singleton-atom cones, then $m - 1$ connecting cones. Insert 2^{2048} empty instructions before and after effective operations and repeat the whole block 2^{4096} times. The correct number of reported events remains $2m - 1$, since all later copies are redundant.

Atoms m	Grammar nodes	Length bits	Events	State trials	Time (ms)
1	6	6146	1	6	0.0217
4	24	6148	7	30	0.1625
8	48	6149	15	62	0.4678
16	96	6150	31	126	1.5982

Table 3: Complete event histories on the sharp-height family, with 2^{2048} empty instructions between effective operations and an outer 2^{4096} repetition.

The $m = 16$ case returns all 31 effective event positions using 126 state trials, with median time about 1.60 ms. Its expanded length has more than 6,000 bits. Independent plateau checking accepts the complete list, while omitting either endpoint event is rejected. This validates an event-history operation rather than merely a final-count shortcut.

Again, the fixture is an explicit algebraic stress family with known expectations. It is not represented as a knot corpus. Large binary powers are essential for testing the intended compression, but their successful handling says nothing about whether a natural knot hierarchy presents the same fixed-atom program structure.

9.4 Native integration gate and promotion criteria

`integration/native_gate.py` is an additive gate for a real ProveIt checkout. It is designed to compare packed and direct-vector native histograms with literal expectations on 200 random small interval systems, check source-bound native certificates in both packing modes, reject source mutations, and test a genuinely compressed eight-atom parallel system with multiplicity 2^{1024} . These are specified future gate checks, not counted as completed experiments in this article.

The saved status is `NOT_RUN`. The missing runnable checkout means that there is no native timing table, no verified maintained-suite count, and no measured whole-knot improvement in this delivery. Before promotion, the gate should pass at the target commit, followed by the maintained test suite and paired native query benchmarks. A production integration should remain opt-in until compilation overhead, cache reuse, and source-refinement frequency have been measured together.

10 Further research questions and proposed next experiments

The most useful follow-up is not to enlarge the benchmark exponents. It is to prove that a concrete geometric consumer uses the fixed-source interface and that the number of recompilations is controlled. The following questions separate those obligations from local engineering work.

10.1 A native source-to-observer contract

For one maintained normal-surface consumer, identify the exact source points, atom cuts, orbit relation, and ownership charges. Prove that every point belongs to the surface component represented by its orbit and that every requested weight is constant on the chosen atoms. Then compare a compiled profile query against the existing direct weighted query, including source validation and independent replay. The success criterion is an actual source-bound equality theorem and measured reuse benefit, not agreement only on a table constructed directly by the benchmark.

10.2 Which geometric operations are genuine full-port cones?

Find a hierarchy subroutine whose aggregate connectivity question really identifies all points in specified port unions. Coning is already a useful auxiliary counting operation, but actual cutting and gluing can preserve pointwise correspondence instead. A precise lemma should state the consumer observation being computed, prove that the coned relation has exactly that meaning, and list all geometric information discarded. A single verified consumer is more valuable than a general assertion that “attachments are unions”.

10.3 A refinement budget

Let a phase introduce new port endpoints. Can those endpoints be charged to a bounded combinatorial resource, so that the total number of distinct atoms and the number of recompilations remain polynomial or quasi-polynomial? The local height proof immediately gives an effective step budget after the last refinement; it gives no such bound beforehand. A useful theorem would bound $\sum_i m_i$ or a sharper persistent-refinement measure across all phases, not merely the largest m_i encountered.

10.4 A richer interface for pointwise attachments

The four-point obstruction identifies missing correlations between point positions inside atoms. What is the weakest efficiently maintainable augmentation that resolves it for interval isometries? Candidates include ordered occurrence maps, compressed permutations, or a profile table augmented with a finite transport action. A proposed augmentation should be tested first on the four-point pair and then on its large repeated analogues. Its description length and composition cost must be bounded independently of expanded multiplicities.

10.5 Program languages with changing labels

The present power rule is sound only for repeated identical cone actions. A natural extension is a grammar node that applies a fixed permutation of atom names between copies. For a supplied finite permutation, one can ask whether its orbit decomposition allows the join of all conjugated cone summaries to be computed by binary powering. The challenge is to preserve exact prefix positions and first-event guarantees when the action is no longer idempotent after one copy. This is a finite algebra question with a clear transition from the current theorem, not an assumed property of geometric monodromy.

10.6 Packing versus direct vector replay

Measure both source compilation modes and direct vector weights on actual native orbit traces. Vary the number of atoms, endpoint bits, support density, and capacity imbalance. Include workloads with no reuse, several low-dimensional queries, and many correlated queries. A possible adaptive policy would use measured arithmetic work to choose between direct replay, trace reuse, and full profile compilation, but it should never substitute a heuristic hash for the exact packing proof. Negative results are useful: large packed integers can be the wrong representation on some traces.

10.7 Sparse profile storage and selected observations

The current reference stores every coordinate in every profile row. Can a sparse row representation reduce sm work while preserving cheap equality, source validation, and quotient aggregation? Conversely, can a requested family of observations be compiled into a smaller exact sufficient statistic than the full atom profile? Theorem 4.3 rules out lossless compression for *all* observations beyond equivalence-preserving encoding, but a restricted observation family may have a much coarser complete quotient. Its closure under the chosen cone action must be proved, not presumed from marginal counts.

10.8 Orientation and additional geometric charges

A fixed signed cover can be compiled separately, but an attachment can introduce new parity constraints. Determine an explicit attachment contract under which cover profiles can be updated without reconstructing the signed relation. Likewise, for Euler or boundary data after a topological move, identify the added and removed cell charges and prove their ownership. The current old-point quotient sum is not a substitute for this topological calculation.

10.9 Formalization and adversarial replay

The finite proofs here are suitable for formalization: mixed-radix decoding, color-preserving orbit equivalence, the quotient graph theorem, the potential $2a - b$, and the prefix/plateau certificate lemmas. A useful formal target would isolate an abstract finite equivalence-relation interface from the encoded AHT layer. On the software side, further fuzzing should target huge serialized inputs, aliases between result wrappers and certificates, and cancellation during native proof construction. Resource exhaustion must remain inconclusive, not become a false topology verdict.

10.10 The global amortization problem

Finally, establish a replacement for the unbounded terms in (5). One promising shape is a decomposition into quasi-polynomially many source-stable phases, each with a succinct operation grammar and a certified interpretation. The necessary theorem must jointly bound phase count, atom refinement, new geometric descriptions and normal vector search. The present $2m - 1$ bound would then remove one repeated local cost automatically. Until those structural bounds are available, it should be used as a proved component of the programme rather than a label for the whole recognizer.

11 Conclusion

A source-atom profile census is more than a collection of marginal counts: it is the exact invariant of an equivalence relation under arbitrary atom-preserving point permutations. That makes it a complete reusable observer for additive atom-constant weights and full-port cones. The multiplicity-aware graph action preserves compressed size, and the active subpartition potential limits a fixed-atom phase to $2m - 1$ effective component changes. This finite-height structure supports exact traversal of binary-repeated programs without traversing their expanded instructions.

The resulting improvement is a rigorous local acceleration with executable reference code, independent replay, and a sharp structural bound. The article also identifies where the abstraction fails and what must be proved before it can accelerate a complete unknot-recognition hierarchy. This combination—an exact improvement and an explicit boundary of validity—is the intended contribution to the repository.

A Artifact map and reproduction

The package is additive. It can be placed under a new research directory without overwriting maintained modules. The suggested location is `Topology/UnknotRecognition/research/compiled_port_quotients/`. The root README gives the exact commands and integration caveats.

Path	Purpose
<code>compiled_ports/profiles.py</code>	Immutable census and exact atom-constant observations.
<code>compiled_ports/packing.py</code>	Mixed-radix and binary-field codecs with capacity checks.
<code>compiled_ports/quotients.py</code>	Lazy multiplicity activation and incremental quotient counts/censuses.
<code>compiled_ports/programs.py</code>	Grammar validation, summaries, prefixes, first thresholds, and all change points.
<code>compiled_ports/verify.py</code>	Independent graph quotient and grammar/plateau replay.
<code>compiled_ports/reference.py</code>	Bounded literal point oracles, used only for finite tests.
<code>compiled_ports/native.py</code>	Adapter to maintained weighted AHT and source-bound native verification.
<code>tests/test_core.py</code>	Executed deterministic, exhaustive, randomized and adversarial tests.
<code>experiments/audit.py</code>	Re-runs the suite and records counters, runtime and source hashes.
<code>experiments/benchmark.py</code>	Seeded controlled algebra-kernel timing experiment.
<code>experiments/make_tables.py</code>	Generates the paper tables from retained JSON data.
<code>integration/native_gate.py</code>	Explicit gate to run against the target native checkout.
<code>integration/source_audit.json</code>	Pinned inspected source paths and Git blob hashes.
<code>results/native_gate_status.json</code>	Explicit record that native execution was not performed here.
<code>examples/</code>	Reproducible large delayed and sharp-height program examples.
<code>paper/main.tex, paper/main.pdf</code>	Complete article source and compiled output.

From the package root:

```
python -m unittest discover -s tests -v
python experiments/audit.py
python experiments/benchmark.py
python experiments/make_tables.py
python examples/demo.py
sh paper/build.sh
```

The first five commands use only Python’s standard library. The build requires a standard LaTeX installation with the packages listed in `paper/main.tex`. Benchmark medians will naturally vary with the host; the exact results and trial counts must not.

For the native gate, supply the maintained `fast` directory:

```
python integration/native_gate.py \
  --fast-root /path/to/ProveIt/Topology/UnknotRecognition/fast
```

A missing native module is a gate error, not a fallback to the literal oracle. The full maintained suite should then be run from that checkout. The gate writes a separate result file, leaving the delivered historical `NOT_RUN` status as a record of this article’s execution scope.

B Minimal usage example

The following example is deliberately an abstract census. Its mass equations are valid, but no source-bound knot or normal-surface claim is implied.

```
from compiled_ports import Profiles, ConeProgram
from compiled_ports.verify import verify_threshold

q = 1 << 256
profiles = Profiles.from_histogram((q,) * 8, {(1,) * 8: q})
nodes = [
    {"op": "cone", "atoms": []},
    {"op": "power", "child": 0, "exponent": 1 << 4096},
    {"op": "cone", "atoms": [3]},
    {"op": "concat", "left": 1, "right": 2},
    {"op": "power", "child": 3, "exponent": 1 << 4096},
]
program = ConeProgram(8, nodes)
answer = program.first_at_most(profiles, 1)
assert answer["index"] == (1 << 4096) + 1
assert answer["trials"] == 2
assert verify_threshold(profiles, nodes, program.root, 1,
                       answer["index"])
```

To obtain a trusted table from an actual native relation, use `compile_profiles(..., record_certificate=True)` and then `verify_native_certificate` against the original pairings and cuts. Only the returned authenticated table should enter a source-bound downstream claim. That path is supplied but awaits the native gate described above.

References

- [1] Ian Agol, Joel Hass, and William Thurston. The computational complexity of knot genus and spanning area. *Transactions of the American Mathematical Society*, 358(9):3821–3850, 2006. DOI: 10.1090/S0002-9947-05-03919-X. Preprint: <https://arxiv.org/abs/math/0205057>. Weighted orbit counting: Theorem 16.
- [2] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. arXiv:2607.23350v1, 2026. <https://arxiv.org/abs/2607.23350>. In particular, Sections 9–10 and Propositions 9.1 and 10.2.
- [3] Robert Endre Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975. DOI: 10.1145/321879.321884.
- [4] ProveIt contributors. *ProveIt: Topology/UnknotRecognition*. GitHub repository owned by Vladimir Reshetnikov, 2026. Inspected baseline commit 1fab6d945be52857cce48b6459075e6194622001. <https://github.com/VladimirReshetnikov/ProveIt/tree/1fab6d945be52857cce48b6459075e6194622001/Topology/UnknotRecognition>.
- [5] ProveIt contributors. Native certified sparse component incidence. Repository synthesis chapter `synthesis/sparse_incidence.tex`, at the baseline in [4]. Git blob 334fd8692883f92cc5b3f5b40fb81fb354e9f463.
- [6] ProveIt contributors. Native weighted normal components and essential-disc counts. Repository synthesis chapter `synthesis/weighted_components.tex`, at the baseline in [4]. Git blob fe2fb0a392f7ff1de5ddbb82e59037dc685a6d14. Inspected with the native weighted producer and its independent verifier; additional hashes are recorded in the artifact manifest.